

What is PitchIQ?

PitchIQ is a sports academy management platform that digitises the way academies track players, communicate with parents, manage trials, and measure performance. Starting with soccer, it expands to rugby, cricket, and beyond — one subscription per school. Built with React, Node.js, Supabase, and Claude AI.

User Roles & Access

■ Admin	Full access · Manage all teams · View all players · Export reports · Invite coaches
■■■ Coach	Manage own team · Log training sessions · Rate players · Add notes & diet plans · Mark bench reasons
■■■■■ Parent	View own child only · See stats, comments, upcoming games, bench reasons, diet plan
■ Player	View own profile · See rank/level · Training history · Upcoming fixtures

Core Modules

1. Player Profiles

Name, age, division (U11–First Team), position, photo, medical info, injury history, diet plan from coach. Access controlled — parents see their child only.

2. Stats & Attendance Tracker

Minutes played per match, training sessions attended, match ratings, goals/assists. Feeds the ranking system and justifies bench decisions to parents.

3. Match & Training Schedule

Upcoming fixtures, lineups, bench lists with reasons. Push notifications eliminate 'when's the game?' messages from parents.

4. Level / Rank System

Rookie → Rising Star → Elite → Master → Grand Master. Calculated from attendance rate + match minutes + coach ratings. Players can show their friends — pure motivation engine.

5. Trial Management

QR code check-in, digital registration form, coach notes per trialist, automated outcome notification to parent. Solves the 800-people-on-paper problem.

6. Coach–Parent Communication

In-app messaging, automated updates: why benched, what to work on at home, diet reminders. No more WhatsApp chaos.

7. Team Leaderboard

Top performers per division, visible within the organisation. Star player spotlight per age group. Drives engagement.

8. Multi-Sport Expansion

Soccer first, then rugby, cricket, chess. Admin adds a sport type and a new team — same structure throughout. Scale to any school.

Tech Stack (Free to Start)

Layer	Tool	Cost
Frontend (web)	React	Free
Mobile App	React Native	Free
Backend	Node.js + Express	Free
Database + Auth	Supabase	Free tier
File Storage	Supabase Storage	Free tier
AI Layer	Claude API (claude-sonnet-4-6)	Pay per use
Hosting	Vercel / Railway	Free tier
Domain	.co.za on Afrihost	~R50/year

Build Sequence

Phase	Module	What Gets Built
Phase 1	Auth System	Login, role-based access, JWT sessions, invite flow
Phase 2	Player Profiles	Profile creation, team/division structure, parent–child link
Phase 3	Stats Logging	Training attendance, match minutes, coach ratings
Phase 4	Schedule + Notifications	Fixtures, lineups, push notifications
Phase 5	Levels / Ranking System	Gamification engine, badges, progress bars
Phase 6	Trial QR Check-in	QR generation, digital registration, outcome notifications
Phase 7	AI Coach Summaries	Claude API integration — auto player summaries
Phase 8	Multi-Sport Expansion	Rugby, cricket, chess — new sport type, same DB schema

Claude Build Prompts

Paste each prompt into Claude Code or claude.ai in order. Complete each phase before moving to the next.

PROMPT 1 — Project Setup & Database Schema

I'm building a sports academy management app called PitchIQ.

Stack: React frontend, Node.js/Express backend, Supabase DB + auth.

Create the full Supabase database schema with these tables:

- users (id, name, email, role: admin|coach|parent|player, created_at)
- organisations (id, name, sport_type, subscription_tier)
- teams (id, org_id, name, division: U11|U12|U13|U14|U15|U16|U19|First_Team, sport)
- players (id, user_id, team_id, position, date_of_birth, medical_notes, photo_url, rank_level)
- parent_player (parent_user_id, player_id)
- training_sessions (id, team_id, coach_id, date, notes)
- attendance (id, session_id, player_id, attended: boolean)
- matches (id, team_id, opponent, date, venue, result)
- player_match_stats (id, player_id, match_id, minutes_played, goals, assists, rating)
- coach_player_notes (id, coach_id, player_id, note, diet_plan, created_at)
- notifications (id, user_id, message, read: boolean, created_at)
- trials (id, org_id, date, sport)
- trial_registrations (id, trial_id, child_name, parent_name, parent_phone, coach_notes, outcome)

Include RLS policies: parents read own child only, coaches update own team only, admins full access.

PROMPT 2 — Auth + Role-Based Access

Build the authentication system for PitchIQ using Supabase Auth and Node.js/Express.

Requirements:

- Registration with role selection (admin, coach, parent, player)
- JWT-based session management
- Middleware that checks role before allowing route access
- Admin can invite coaches via email link
- Parent links to their child profile using a unique child code from admin
- Route protection: /admin/* admins only, /coach/* coaches+admins, /parent/* parents only

Use Express Router. Return clear error messages on denied access.

PROMPT 3 — Player Profile + Rank System UI

Build the Player Profile page for PitchIQ in React.

Show: player photo, name, position, age, division/team, rank badge with progress bar, stats summary (training attendance, match minutes, avg rating), last 3 coach notes, diet plan checklist, next upcoming event, medical flag (visible to coach/parent/admin only).

Rank levels: Rookie / Rising Star / Elite / Master / Grand Master

Calculated from: attendance rate + total match minutes + avg coach rating (1-5)

Dark theme, green accent #00e676, clean card layout, mobile responsive.

PROMPT 4 — Coach Dashboard & Stats Logging

Build the Coach Dashboard for PitchIQ in React + Node.js API.

Coach must be able to:

1. Log training session – date, attendance per player, session note
2. Log match – opponent, date, result, then per player: minutes, goals, assists, rating, note
3. Add note or diet plan to a specific player
4. View squad list with attendance rate and avg rating per player
5. Mark a player as benched with a reason (visible to parent)

Build Express routes:

POST /coach/training-session

POST /coach/match-log

PATCH /coach/player/:id/notes

GET /coach/squad/:team_id

Mobile-first UI – coaches log this on a phone after training.

PROMPT 5 — Trial Management with QR Code

```
Build the Trial Day module for PitchIQ.

Admin: create trial event, generate unique QR code, view all registrations table.

Parent/public (no login): scan QR -> mobile registration form
Fields: child name, age, position, parent name, phone, email
On submit: save to trial_registrations, show confirmation screen.

Coach: view trialists for their session, add notes, mark outcome: accepted/declined/pending

When outcome is marked, parent gets automated message:
"Hi [parent], we have reviewed [child]'s trial. [Outcome]. - PitchIQ / [Academy]"

Use qrcode npm package. Mobile-first design.
```

PROMPT 6 — AI Coach Summary (Claude API)

```
Add an AI Player Summary feature to PitchIQ using the Claude API.

When coach clicks "Generate AI Summary" on a player profile:
1. Pull last 5 training sessions, last 3 match stats, last 3 coach notes from DB
2. Send to Claude API (claude-sonnet-4-6) with this system prompt:

"You are a youth sports development assistant for PitchIQ. Given a player's recent
stats and coach notes, write a short encouraging but honest performance summary
(max 150 words). Include: what the player is doing well, one area to improve,
one practical home training tip. Write for both parent and a 13-year-old to understand."

3. Display summary on player profile under "AI Summary - Last 30 Days"
4. Include a Regenerate button that pulls fresh data and calls the API again

Handle loading and error states. Use fetch to call the Claude API.
```

How to Start Today — Zero Capital

- 1** **supabase.com**
Create free project — this is your database
- 2** **Claude Code / claude.ai**
Paste Prompt 1 above, build the schema
- 3** **vercel.com**
Free hosting for your frontend
- 4** **Afrihost .co.za**
Register pitchiq.co.za — under R100/year
- 5** **One School**
Walk into Tux Academy with Phase 1+2 working on your phone. That's your pilot.